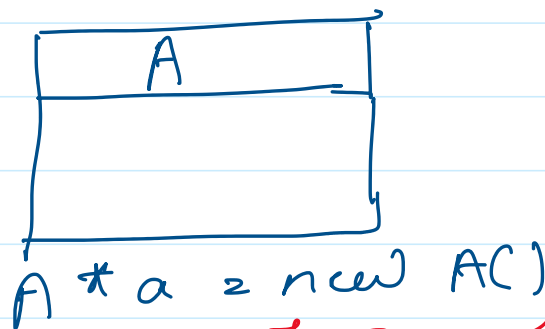


Singleton

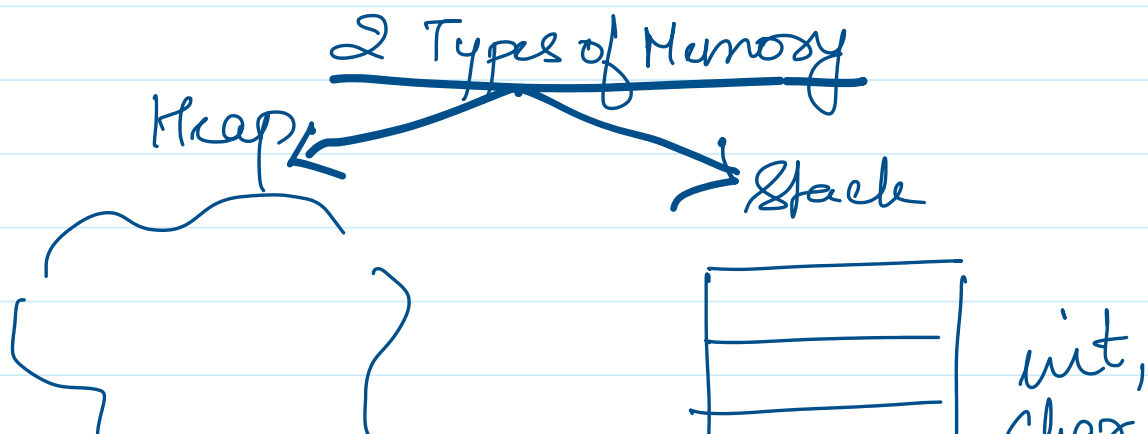
It is a type of Class whose only one object can be created

When you try to make another object it will return the instance of previous object. So, not more than one object is allowed.

Let's try to make such class.

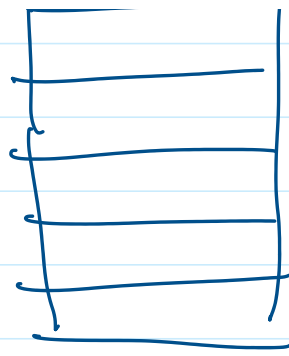


Let's understand what happens when 'new' is used





Here all non-primitive data type is stored



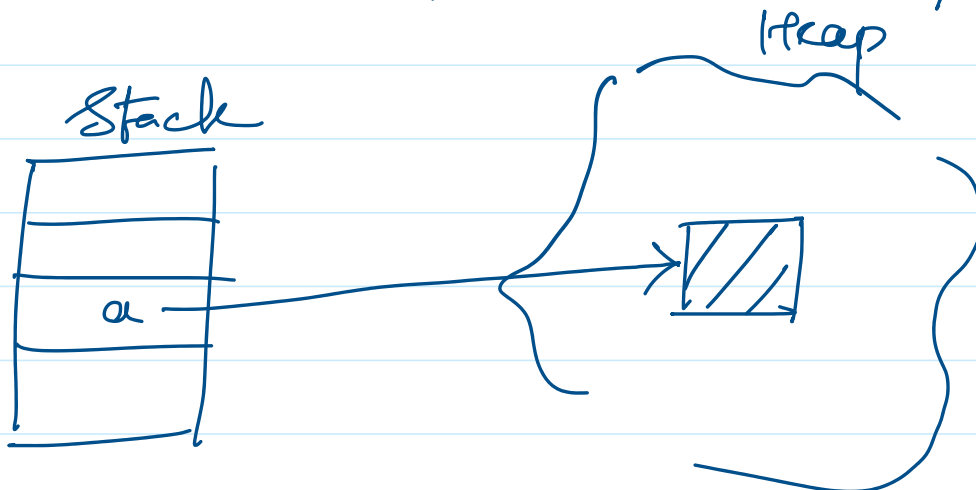
int,
char,
bool.

Here all primitive data type is stored.

What happens when

$A^* a_2 = \text{new AC}()$ is called?

1. In heap a space is reserved where our object gets the memory
2. And a_n ^{in stack} $\rightarrow$ points to that space



Now the whole culprit is "new" keyword, we can create as many object in heap by $\downarrow$

$A^* a_2 = \text{new AC}();$
 $a_1 \quad \quad \quad a_2$

```
A* a2 = new A();  
A* a3 = new A();  
:  
:
```

Now how we can stop creating new object - let's see in code.

Code - to make singleton class where you can create object only once
→ see Notes...

SimpleSingleton.java

Now in code you can see 2 thread can enter simultaneously inside if statement and both can make their own object of singleton class
∴ 2 objects created & our singleton logic fails

To prevent this we can introduce locks in our code

Code - to make singleton class with locks → see Notes...

ThreadSafeLockingSingleton.java

But we know locking is a very

But we know locking is a very expensive thing. It reserves all the resources. So, it's better to avoid this.

So, we first check whether the instance is 'null' or not. If it is not 'null' then other thread will not go inside, so, locking is not required.

Code for locking when required
→ see Notes...

ThreadSafe Doublelocking Singleton.
java

There is one more way where we don't require any lock.

Code → see Notes...

ThreadSafe Eager Singleton.java

↳ Disadvantage:-

- Suppose this class is very expensive to create it lot of memory is used
- So you creating the object of

- So you creating the object of such class even before the main function starts running
 - Now suppose method of such singleton class never used
 - So, you are wasting memory
- ∴ Eager Initialization is not preferred
- used only if such class is light-weight, not so expensive.

Conclusion:-

→ Create a private constructor

→ Create a static instance (getInstance) that returns the same instance every time.

Real world Usage

1. Logging System -

for entire application we need only one object of "logger" class to maintain consistency

Suppose each & every service make their own "logger" object then logs will print up & down, no sequencing.

It will be difficult to manage so many object of logger.

∴ we need only one object of logger for whole application

2. Database Connection.

Database connection in our application is a very expensive operation. lot of memory & space is consumed in this.

So, for whole application we need one object which connect our application to Database. And we use that connection in whatever part of application we want.

3. Configuration Manager.

Suppose we have APIs in our application and to access it we have API keys.

And we store those API keys in some configuration file.

Now we want all services to hold a single object of that configuration

If they hold different object than anyone can change the configuration

So, to maintain Single Source of Truth we have to make it singleton.

we have to make it "singleton".

Where Singleton is Not Used :-

You know there is class where you need multiple instance of Class like Game :-

↳ where different players are there who interacts. So every player is object, they should not share object

Note :-

In Singleton it is important to write thread safe code. Therefore more complication arises b/c of thread.

So, to write Unit testing becomes difficult

So, try to avoid Singleton till you know it is not going to help us anymore.

But use it in cases discussed above b/c

- It saves memory
- Single Source of Truth.